7.4 Attack Simulator

To assess robustness, we provide an attack suite that modifies the stegotext in realistic ways:

- **n-Gram Shuffling:** Randomly shuffles small text segments, measuring how the watermarking scheme and embedding-based approach handle attacks such as reordering sections of text.
- **Synonym Substitution:** Replaces words with synonyms. Users can tune the fraction of tokens replaced, balancing realism and severity.
- **AI Paraphrasing:** Uses an LLM to paraphrase sentences (local) or larger blocks of text (global), reflecting scenarios in which adversaries attempt to preserve semantic meaning while altering how the message is phrased.

After each attack, the stegotext is fed into the decoding function to measure recovered message rates.

7.5 Performance Evaluation

We conducted experiments to measure each system's performance, covertext overhead, and robustness against attacks. Our primary metrics include frequency of recurring n-grams within a text, how parameters affect covertext length, message recovery accuracy, rejection sampling efficiency, embedding drift under attacks, and the cost per hidden bit when using paid APIs.

Watermarking-Based System

Recurring c-grams We measure how frequently c-grams occur in generated text as c increases (Figure 3). Intuitively, higher values of c repeat less frequently, leaving little to be learned by adversaries. All values $c > 2$ occur at low rates in covertexts of 100 tokens.

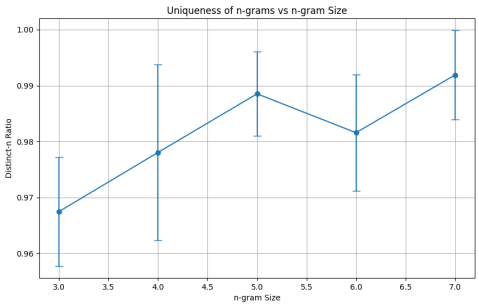


Fig. 3: Frequency of recurring c-grams is measured in 100 samples of text containing 100 tokens sampled from GPT-2. As the size of c increases, duplicates are less likely to be found.

δ Effect on Covertext Length We measure how varying δ impacts covertext length (Figure 4). We find that as δ increases, the required covertext length decreases. The trade-off lies in the fact that larger values of delta disrupt the "naturalness" of generated text, with high values violating the stealthyness of the scheme.

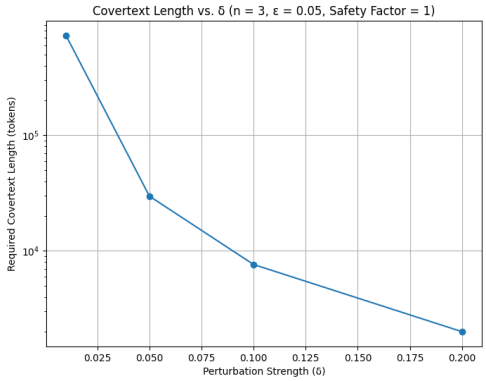


Fig. 4: δ's effect on covertext length. Keeping a fixed $n = 3$ and safety factor of 1, we measure the necessary length of the covertext to successfully hide messages with 95% accuracy.

Embedding-Based System

Rejection Sampling Efficiency. We measure how many samples (on average) are needed to find a contextually relevant message whose embedding hashes to the desired value (Figure 5). We test this for both the Random LSH and the PCA LSH for 1 bit outputs and simulate trials for multi-bit outputs (see Figure 5). This was done over 100 trials and we did not cap the amount of attempts. We found that the Random LSH took 2.82 samples on average over the 100 trials. This differs from the expected 2^n hashes due to the next likely tokens being biased to one side of the random cut. The PCA LSH took 2.03 samples on average over 100 trials. The corporate prompt can be found at Appendix D. This corresponds to corporate emails from the Enron dataset and was specifically made for the PCA LSH.

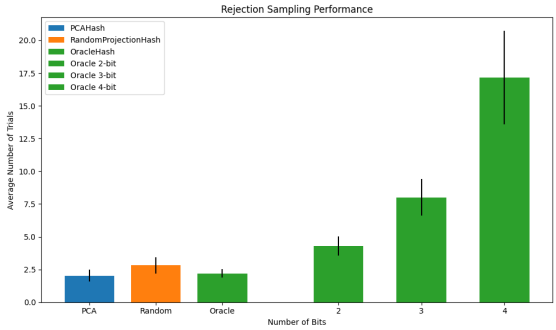
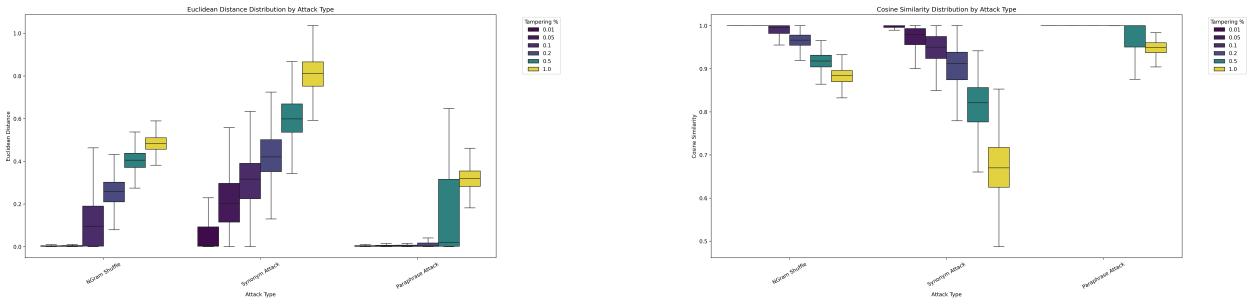


Fig. 5: Measurements of embeddings rejection sampling. The Random and PCA LSH functions hash to 1 bit and used a conversational and corporate conversation mode respectively. The corporate mode is designed for the PCA LSH. LSH functions that output more than 1 bit were simulated via the Oracle LSH.

Embedding Drift. We measure how various attacks impact the similarity (Euclidean distance and cosine similarity) of embeddings before and after the attack (Figure 6). This helps inform which LSH functions can resist certain attack types. As expected, increasing the percentage of the text that is attacked creates further distance between the pair of text– although n-gram shuffle attacks and paraphrase attacks remain close even at high percentages. Surprisingly, the synonym attack is what causes large changes. This is due to nltk's wordnet being used for the attack and making overly aggressive substitutions. These substitutions include choices that are not grammatically correct, such as changing "not" to "non" or words that change the meaning of sentences. Therefore, many synonym attacks fall outside the scope of the strong robustness threat model as they are not changes that the adversary would actually want to make and are therefore not a source of concern.



(a) Euclidean distance for various attack categories and percentage of text attacked.

(b) Cosine similarity for various attack categories and percentage of text attacked.

Fig. 6

Cost per Hidden Bit. Here we first derive the cost of hiding an n bit message and then empirically test the cost with OpenAI's API using `gpt-4o-mini` and `text-embedding-3-large`. We define the variables as follows:

- n: Total number of bits to hide.
- h: Number of bits hidden per successful covertext chunk (i.e., the hash output size).
- c: Average number of queries needed per successful chunk (for a uniform hash, $c \approx 2^h$).
- W: Number of input tokens (context window tokens) per query.
- T: Number of output tokens (covertext chunk tokens) per query.
- p_{in}: Cost per input token.
- p_{out}: Cost per output token.

Each successful chunk hides h bits, so the number of successful chunks required is

$$\text{Successful Chunks} = \frac{n}{h}.$$

Since each chunk requires c queries on average, the total number of queries is

$$\text{Total Queries} = \frac{n}{h} \cdot c.$$

The cost per query is given by

$$\text{Cost per Query} = W \cdot p_{\text{in}} + T \cdot p_{\text{out}}.$$